CRIME REPORTING SYSTEM-CASE STUDY

ENTITY PACKAGE:

1.Evidence Class

```
package entity;
public class Evidence {
    private String evidenceID;
    private String description;
    private String locationFound;
    private int incidentID;

    // Default constructor
    public Evidence() {}

    // Parameterized constructor
    public Evidence(String evidenceID, String description, String
locationFound, int incidentID) {
        this.evidenceID = evidenceID;
        this.description = description;
        this.locationFound = locationFound;
        this.incidentID = incidentID;
    }

    // Getters and Setters
    public String getEvidenceID() {
        return evidenceID;
```

```
}

public void setEvidenceID(String evidenceID) {
    this.evidenceID = evidenceID;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public String getLocationFound() {
    return locationFound;
}

public void setLocationFound(String locationFound) {
    this.locationFound = locationFound;
}

public int getIncidentID() {
    return incidentID;
}
```

```
    public void setIncidentID(int incidentID) {
        this.incidentID = incidentID;
    }

    // toString method
    @Override
    public String toString() {
        return "Evidence{" +
                "evidenceID='" + evidenceID + '\'' +
                ", description='" + description + '\'' +
                ", locationFound='" + locationFound + '\'' +
                ", incidentID=" + incidentID +
                '}';
    }
}
```

2.Incident Class

```
package entity;
import java.util.Date;
public class Incident {
    private int incidentID;
    private String incidentType;
    private Date incidentDate;
    private double latitude;
    private double longitude;
    private String description;
    private String status;
```

```
    private int victimID;
    private String suspectID;
    private String officerID;

    // Default Constructor
    public Incident() {}

    // Parameterized Constructor
    public Incident(int incidentID, String incidentType, Date
incidentDate, double latitude, double longitude, String description,
String status, int victimID, String suspectID, String officerID) {
        this.incidentID = incidentID;
        this.incidentType = incidentType;
        this.incidentDate = incidentDate;
        this.latitude = latitude;
        this.longitude = longitude;
        this.description = description;
        this.status = status;
        this.victimID = victimID;
        this.suspectID = suspectID;
        this.officerID = officerID;
    }

    // Getters and Setters
    public int getIncidentID() {
        return incidentID;
```

```
}

public void setIncidentID(int incidentID) {
    this.incidentID = incidentID;
}

public String getIncidentType() {
    return incidentType;
}

public void setIncidentType(String incidentType) {
    this.incidentType = incidentType;
}

public Date getIncidentDate() {
    return incidentDate;
}

public void setIncidentDate(Date incidentDate) {
    this.incidentDate = incidentDate;
}

public double getLatitude() {
    return latitude;
}
```

```
public void setLatitude(double latitude) {
    this.latitude = latitude;
}

public double getLongitude() {
    return longitude;
}

public void setLongitude(double longitude) {
    this.longitude = longitude;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public String getStatus() {
    return status;
}

public void setStatus(String status) {
    this.status = status;
```

```
}

public int getVictimID() {
    return victimID;
}

public void setVictimID(int victimID) {
    this.victimID = victimID;
}

public String getSuspectID() {
    return suspectID;
}

public void setSuspectID(String suspectID) {
    this.suspectID = suspectID;
}

public String getOfficerID() {
    return officerID;
}

public void setOfficerID(String officerID) {
    this.officerID = officerID;

}
```

```
    @Override
    public String toString() {
        return "Incident ID: " + incidentID +
                ", Type: " + incidentType +
                ", Date: " + incidentDate +
                ", Location: (" + latitude + ", " + longitude + ")" +
                ", Description: " + description +
                ", Status: " + status +
                ", Victim ID: " + victimID +
                ", Suspect ID: " + suspectID +
                ", Officer ID: " + officerID;
    }
}
```

3.Suspects Class

```
package entity;

import java.time.LocalDate;

public class Suspect {
    private String suspectID;
    private String firstName;
    private String lastName;
    private LocalDate dateOfBirth;
    private String gender;
    private String address;
    private String phoneNumber;
```

```
    // Default constructor
    public Suspect() {}

    // Parameterized constructor
    public Suspect(String suspectID, String firstName, String lastName,
LocalDate dateOfBirth, String gender, String address, String
phoneNumber) {
        this.suspectID = suspectID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.dateOfBirth = dateOfBirth;
        this.gender = gender;
        this.address = address;
        this.phoneNumber = phoneNumber;
    }

    // Getters and Setters
    public String getSuspectID() {
        return suspectID;
    }

    public void setSuspectID(String suspectID) {
        this.suspectID = suspectID;
    }
```

```
public String getFirstName() {
    return firstName;
}

public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public String getLastName() {
    return lastName;
}

public void setLastName(String lastName) {
    this.lastName = lastName;
}

public LocalDate getDateOfBirth() {
    return dateOfBirth;
}

public void setDateOfBirth(LocalDate dateOfBirth) {
    this.dateOfBirth = dateOfBirth;
}

public String getGender() {
    return gender;
```

```
}

public void setGender(String gender) {
    this.gender = gender;
}

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

public String getPhoneNumber() {
    return phoneNumber;
}

public void setPhoneNumber(String phoneNumber) {
    this.phoneNumber = phoneNumber;
}

// toString method
@Override
public String toString() {
    return "Suspect{" +
```

```
                "suspectID='" + suspectID + '\'' +
                ", firstName='" + firstName + '\'' +
                ", lastName='" + lastName + '\'' +
                ", dateOfBirth=" + dateOfBirth +
                ", gender='" + gender + '\'' +
                ", address='" + address + '\'' +
                ", phoneNumber='" + phoneNumber + '\'' +
                '}';
    }
}
```

4.Reports Class

```
package entity;

import java.time.LocalDateTime;

public class Reports {
    private int reportID;
    private int incidentID;
    private String reportingOfficer;
    private LocalDateTime reportDate;
    private String reportDetails;
    private String status;

    // Default constructor
    public Reports() {}
```

```
    // Parameterized constructor
    public Reports(int reportID, int incidentID, String reportingOfficer,
LocalDateTime reportDate, String reportDetails, String status) {
        this.reportID = reportID;
        this.incidentID = incidentID;
        this.reportingOfficer = reportingOfficer;
        this.reportDate = reportDate;
        this.reportDetails = reportDetails;
        this.status = status;
    }

    // Getters and Setters
    public int getReportID() {
        return reportID;
    }

    public void setReportID(int reportID) {
        this.reportID = reportID;
    }

    public int getIncidentID() {
        return incidentID;
    }

    public void setIncidentID(int incidentID) {
        this.incidentID = incidentID;
```

```
}

public String getReportingOfficer() {
    return reportingOfficer;
}

public void setReportingOfficer(String reportingOfficer) {
    this.reportingOfficer = reportingOfficer;
}

public LocalDateTime getReportDate() {
    return reportDate;
}

public void setReportDate(LocalDateTime reportDate) {
    this.reportDate = reportDate;
}

public String getReportDetails() {
    return reportDetails;
}

public void setReportDetails(String reportDetails) {
    this.reportDetails = reportDetails;
}
```

```
    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    // toString method
    @Override
    public String toString() {
        return "Report{" +
                "reportID=" + reportID +
                ", incidentID=" + incidentID +
                ", reportingOfficer='" + reportingOfficer + '\'' +
                ", reportDate=" + reportDate +
                ", reportDetails='" + reportDetails + '\'' +
                ", status='" + status + '\'' +
                '}';
    }
}
```

5.Victims Class

```
package entity;

import java.util.Date;
```

```
public class Victim {
    private int victimID;
    private String firstName;
    private String lastName;
    private Date dateOfBirth;
    private String gender;
    private String address;
    private String phoneNumber;

    // Default Constructor
    public Victim() {}

    // Parameterized Constructor
    public Victim(int victimID, String firstName, String lastName, Date
dateOfBirth, String gender, String address, String phoneNumber) {
        this.victimID = victimID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.dateOfBirth = dateOfBirth;
        this.gender = gender;
        this.address = address;
        this.phoneNumber = phoneNumber;
    }

    // Getters and Setters
    public int getVictimID() {
```

```
    return victimID;
}

public void setVictimID(int victimID) {
    this.victimID = victimID;
}

public String getFirstName() {
    return firstName;
}

public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public String getLastName() {
    return lastName;
}

public void setLastName(String lastName) {
    this.lastName = lastName;
}

public Date getDateOfBirth() {
    return dateOfBirth;
}
```

```
public void setDateOfBirth(Date dateOfBirth) {
    this.dateOfBirth = dateOfBirth;
}

public String getGender() {
    return gender;
}

public void setGender(String gender) {
    this.gender = gender;
}

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

public String getPhoneNumber() {
    return phoneNumber;
}

public void setPhoneNumber(String phoneNumber) {
```

```
        this.phoneNumber = phoneNumber;
    }
    @Override
    public String toString() {
        return "Victim{" +
                "victimID=" + victimID +
                ", firstName='" + firstName + '\'' +
                ", lastName='" + lastName + '\'' +
                ", dateOfBirth=" + dateOfBirth +
                ", gender='" + gender + '\'' +
                ", address='" + address + '\'' +
                ", phoneNumber='" + phoneNumber + '\'' +
                '}';
    }

}
```

6.Officers

```
package entity;

public class Officers {
    private String officerID;
    private String firstName;
    private String lastName;
    private String badgeNumber;
    private String rank;
```

```
    private String contactInfo;
    private String agencyID;
    //Default Constructor
    public Officers(){}
    // Constructor
    public Officers(String officerID, String firstName, String lastName,
String badgeNumber, String rank, String contactInfo, String agencyID) {
        this.officerID = officerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.badgeNumber = badgeNumber;
        this.rank = rank;
        this.contactInfo = contactInfo;
        this.agencyID = agencyID;
    }

    // Getters and Setters
    public String getOfficerID() {
        return officerID;
    }

    public void setOfficerID(String officerID) {
        this.officerID = officerID;
    }

    public String getFirstName() {
```

```
    return firstName;
}

public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public String getLastName() {
    return lastName;
}

public void setLastName(String lastName) {
    this.lastName = lastName;
}

public String getBadgeNumber() {
    return badgeNumber;
}

public void setBadgeNumber(String badgeNumber) {
    this.badgeNumber = badgeNumber;
}

public String getRank() {
    return rank;
}
```

```
public void setRank(String rank) {
    this.rank = rank;
}

public String getContactInfo() {
    return contactInfo;
}

public void setContactInfo(String contactInfo) {
    this.contactInfo = contactInfo;
}

public String getAgencyID() {
    return agencyID;
}

public void setAgencyID(String agencyID) {
    this.agencyID = agencyID;
}

// toString method for debugging
@Override
public String toString() {
    return "Officer{" +
            "officerID='" + officerID + '\'' +
```

```
                ", firstName='" + firstName + '\'' +
                ", lastName='" + lastName + '\'' +
                ", badgeNumber='" + badgeNumber + '\'' +
                ", rank='" + rank + '\'' +
                ", contactInfo='" + contactInfo + '\'' +
                ", agencyID='" + agencyID + '\'' +
                '}';
    }
}
```

7.JunctionIV Class

```
package entity;

public class JunctionIV {
    private int incidentID;
    private String victimID;

    // Default constructor
    public JunctionIV() {}

    // Parameterized constructor
    public JunctionIV(int incidentID, String victimID) {
        this.incidentID = incidentID;
        this.victimID = victimID;
    }

    // Getters and Setters
```

```
    public int getIncidentID() {
        return incidentID;
    }

    public void setIncidentID(int incidentID) {
        this.incidentID = incidentID;
    }

    public String getVictimID() {
        return victimID;
    }

    public void setVictimID(String victimID) {
        this.victimID = victimID;
    }

    // toString method

    @Override
    public String toString() {
        return "JunctionIV{" +
                "incidentID=" + incidentID +  // Corrected: Removed extra
quotes
                ", victimID='" + victimID + '\'' +
                '}';
    }
```

```
}
```

8.JunctionIS Class:

```
package entity;

public class JunctionIS {
    private int incidentID;
    private String suspectID;

    // Default constructor
    public JunctionIS() {}

    // Parameterized constructor
    public JunctionIS(int incidentID, String suspectID) {
        this.incidentID = incidentID;
        this.suspectID = suspectID;
    }

    // Getters and Setters
    public int getIncidentID() {
        return incidentID;
    }

    public void setIncidentID(int incidentID) {
        this.incidentID = incidentID;
```

```
    }

    public String getSuspectID() {
        return suspectID;
    }

    public void setSuspectID(String suspectID) {
        this.suspectID = suspectID;
    }

    // toString method
    @Override
    public String toString() {
        return "JunctionIS{" +
                "incidentID=" + incidentID +
                ", suspectID='" + suspectID + '\'' +
                '}';
    }
}
```

UTIL PACKAGE:

1.DBConnection Class

```
package util;

import java.sql.Connection;
import java.sql.DriverManager;
```

```
import java.sql.SQLException;

public class DBConnection {

    // Static method that takes a connection string and returns a database connection
    public static Connection getConnection(String connectionString) {
        if (connectionString == null || connectionString.isEmpty()) {
            System.err.println("Connection string is null or empty!");
            return null;
        }

        try {
            // Load JDBC driver (if necessary)
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection conn = DriverManager.getConnection(connectionString);
            System.out.println("Database connected!");
            return conn;

        } catch (ClassNotFoundException e) {
            System.err.println("JDBC Driver not found!");
            e.printStackTrace();
        } catch (SQLException e) {
            System.err.println("Database connection failed: " + e.getMessage());
```

```
            e.printStackTrace();
        }
        return null;
    }
}
```

2.DBPropertyUtil Class

```
package util;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

public class DBPropertyUtil {

    // Method to load properties file and return the connection string
    public static String getConnectionString(String fileName) {
        Properties properties = new Properties();

        // Load the properties file from resources
        try (InputStream input =
DBPropertyUtil.class.getClassLoader().getResourceAsStream(fileName)
) {
            if (input == null) {
                System.err.println("Unable to find " + fileName);
                return null;
            }
```

```
            properties.load(input);
        } catch (IOException e) {
            System.err.println("Failed to load properties file " + fileName);
            e.printStackTrace();
            return null;
        }

        // Extract database properties
        String url = properties.getProperty("db.url");
        String user = properties.getProperty("db.username");
        String password = properties.getProperty("db.password");

        if (url == null || user == null || password == null) {
            System.err.println("Missing required database properties in " +
fileName);
            return null;
        }

        // Return the formatted connection string
        return url + "?user=" + user + "&password=" + password;
    }
}
```

DAO PACKAGE:

```
package dao;
```

```
import
com.hexaware.myexceptions.IncidentNumberNotFoundException;
import entity.Incident;
import entity.Reports;
import java.sql.SQLException;
import java.util.Collection;
import java.util.Date;

public interface ICrimeAnalysisService {

    boolean createIncident(Incident incident) throws SQLException;

    boolean updateIncidentStatus(int incidentID, String status) throws
SQLException, IncidentNumberNotFoundException;

    Collection<Incident> getIncidentsInDateRange(Date startDate, Date
endDate) throws SQLException;

    Collection<Incident> searchIncidents(String incidentType) throws
SQLException;

    Reports generateIncidentReport(int reportID, Incident incident, String
reportDetails) throws SQLException;

    boolean reportExists(int reportID) throws SQLException;

    boolean finalizeReport(int reportID) throws SQLException;
```

```
}
```

2. CrimeAnalysisServiceImpl Class

```
package dao;

import
com.hexaware.myexceptions.IncidentNumberNotFoundException;

import entity.Incident;

import entity.Reports;

import java.sql.*;

import java.time.LocalDateTime;

import java.util.ArrayList;

import java.util.Collection;

import java.util.Date;

import util.DBConnection;

import util.DBPropertyUtil;


public class CrimeAnalysisServiceImpl implements
ICrimeAnalysisService {


    private static final String PROPERTIES_FILE =
"dbconfig.properties";
    private static Connection connection;


    // Constructor to initialize the connection
    public CrimeAnalysisServiceImpl() {
        if (connection == null) {
```

```
            String connectionString =
DBPropertyUtil.getConnectionString(PROPERTIES_FILE);
            connection = DBConnection.getConnection(connectionString);
        }
    }

    // Create a new incident
    @Override
    public boolean createIncident(Incident incident) throws
SQLException {
        String sql = "INSERT INTO Incidents (IncidentID, IncidentType,
IncidentDate, Latitude, Longitude, Description, Status, VictimID,
SuspectID, OfficerID) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, incident.getIncidentID());
            ps.setString(2, incident.getIncidentType());
            ps.setDate(3, new
java.sql.Date(incident.getIncidentDate().getTime()));
            ps.setDouble(4, incident.getLatitude());
            ps.setDouble(5, incident.getLongitude());
            ps.setString(6, incident.getDescription());
            ps.setString(7, incident.getStatus());
            ps.setInt(8, incident.getVictimID());
            ps.setString(9, incident.getSuspectID());  // Ensure consistency in
data type
            ps.setString(10, incident.getOfficerID());
```

```
            return ps.executeUpdate() > 0;
        }
    }

    // Update the status of an incident
    @Override
    public boolean updateIncidentStatus(int incidentID, String status)
throws SQLException, IncidentNumberNotFoundException {
        String sql = "UPDATE Incidents SET Status = ? WHERE
IncidentID = ?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, status);
            ps.setInt(2, incidentID);

            int rowsUpdated = ps.executeUpdate();
            if (rowsUpdated == 0) {
                throw new IncidentNumberNotFoundException("Incident ID "
+ incidentID + " not found in the database.");
            }
            return true;
        }
    }

    public Collection<Incident> getIncidentsInDateRange(Date startDate,
Date endDate) throws SQLException {
        Collection<Incident> incidentList = new ArrayList<>();
```

```
        String sql = "SELECT * FROM Incidents WHERE IncidentDate
BETWEEN ? AND ?";

        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setDate(1, new java.sql.Date(startDate.getTime()));
            ps.setDate(2, new java.sql.Date(endDate.getTime()));
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                Incident incident = new Incident(
                    rs.getInt("IncidentID"),
                    rs.getString("IncidentType"),
                    rs.getTimestamp("IncidentDate"),
                    rs.getDouble("Latitude"),
                    rs.getDouble("Longitude"),
                    rs.getString("Description"),
                    rs.getString("Status"),
                    rs.getInt("VictimID"),
                    rs.getString("SuspectID"),
                    rs.getString("OfficerID")
                );
                incidentList.add(incident);
            }
        }
        return incidentList;
    }
```

```
    // Search for incidents based on incident type
    @Override
    public Collection<Incident> searchIncidents(String incidentType)
throws SQLException {
        String sql = "SELECT * FROM Incidents WHERE IncidentType =
?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, incidentType);


            try (ResultSet rs = ps.executeQuery()) {
                Collection<Incident> incidents = new ArrayList<>();
                while (rs.next()) {
                    incidents.add(new Incident(
                        rs.getInt("IncidentID"),
                        rs.getString("IncidentType"),
                        rs.getDate("IncidentDate"),
                        rs.getDouble("Latitude"),
                        rs.getDouble("Longitude"),
                        rs.getString("Description"),
                        rs.getString("Status"),
                        rs.getInt("VictimID"),
                        rs.getString("SuspectID"),  // Ensure correct type
                        rs.getString("OfficerID")
                    ));
```

```
            }
            return incidents;
        }
    }
}
@Override
public boolean reportExists(int reportID) throws SQLException {
    String sql = "SELECT COUNT(*) FROM Reports WHERE
ReportID = ?";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, reportID);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return rs.getInt(1) > 0;
        }
    }
    return false;
}


@Override
public Reports generateIncidentReport(int reportID, Incident incident,
String reportDetails) throws SQLException {
    if (reportExists(reportID)) {
        System.out.println("Error: Report ID already exists. Please use a
new ID.");
        return null;
    }
```

```
        String sql = "INSERT INTO Reports (ReportID, IncidentID,
ReportingOfficer, ReportDetails, Status, ReportDate) VALUES (?, ?, ?,
?, ?, ?)";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, reportID);
            ps.setInt(2, incident.getIncidentID());
            ps.setString(3, incident.getOfficerID());
            ps.setString(4, reportDetails);
            ps.setString(5, "Draft");
            ps.setTimestamp(6, Timestamp.valueOf(LocalDateTime.now()));

            int affectedRows = ps.executeUpdate();
            if (affectedRows > 0) {
                return new Reports(reportID, incident.getIncidentID(),
incident.getOfficerID(),
                    LocalDateTime.now(), reportDetails, "Draft");
            }
        }
        return null;
    }

    public boolean finalizeReport(int reportID) throws SQLException {
        String sql = "UPDATE Reports SET Status = 'Finalized' WHERE
ReportID = ? AND Status = 'Draft'";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, reportID);
```

```
            return ps.executeUpdate() > 0;
        }
    }
}
```

com.hexaware.myexceptions PACKAGE:

1. IncidentNumberNotFoundException Class

```
package com.hexaware.myexceptions;
public class IncidentNumberNotFoundException extends Exception {
    public IncidentNumberNotFoundException(String message) {
        super(message);
    }
}
```

Main Package:

1.MainModule Class

```
package main;

import
com.hexaware.myexceptions.IncidentNumberNotFoundException;
import dao.CrimeAnalysisServiceImpl;
import entity.Incident;
import entity.Reports;
import java.sql.SQLException;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Collection;
```

```
import java.util.Date;
import java.util.Scanner;

public class MainModule {
    public static void main(String[] args) {
        CrimeAnalysisServiceImpl service = new
CrimeAnalysisServiceImpl();
        Scanner scanner = new Scanner(System.in);
        boolean exit = false;

        while (!exit) {
            System.out.println("\nCrime Analysis System");
            System.out.println("1. Create Incident");
            System.out.println("2. Update Incident Status");
            System.out.println("3. Get Incidents in Date Range");
            System.out.println("4. Search Incidents by Type");
            System.out.println("5. Generate Incident Report");
            System.out.println("6. Finalize Report");
            System.out.println("7. Exit");
            System.out.print("Enter your choice: ");

            int choice = scanner.nextInt();
            scanner.nextLine(); // Consume newline

            switch (choice) {
                case 1:
```

```
try {
    System.out.print("Enter Incident ID: ");
    int incidentID = scanner.nextInt();
    scanner.nextLine();

    System.out.print("Enter Incident Type: ");
    String incidentType = scanner.nextLine();

    System.out.print("Enter Latitude: ");
    double latitude = scanner.nextDouble();
    System.out.print("Enter Longitude: ");
    double longitude = scanner.nextDouble();
    scanner.nextLine();

    System.out.print("Enter Description: ");
    String description = scanner.nextLine();

    System.out.print("Enter Status: ");
    String status = scanner.nextLine();

    System.out.print("Enter Victim ID: ");
    int victimID = scanner.nextInt();
    scanner.nextLine();

    System.out.print("Enter Suspect ID: ");
    String suspectID = scanner.nextLine();
```

```
                System.out.print("Enter Officer ID: ");
                String officerID = scanner.nextLine();

                Incident incident = new Incident(incidentID,
incidentType, new Date(), latitude, longitude, description, status,
victimID, suspectID, officerID);
                boolean created = service.createIncident(incident);
                System.out.println(created ? "Incident Created
Successfully!" : "Failed to Create Incident");

            } catch (SQLException e) {
                System.out.println("Error: " + e.getMessage());
            }
            break;

        case 2:
            try {
                System.out.print("Enter Incident ID to Update: ");
                int updateIncidentID = scanner.nextInt();
                scanner.nextLine();

                System.out.print("Enter New Status: ");
                String newStatus = scanner.nextLine();

                boolean updated =
service.updateIncidentStatus(updateIncidentID, newStatus);
```

```
                System.out.println(updated ? "Incident Status Updated!" :
"Update Failed");

            } catch (IncidentNumberNotFoundException e) {
                System.out.println("Error: " + e.getMessage());
            } catch (SQLException e) {
                System.out.println("Database Error: " + e.getMessage());
            }
            break;

        case 3:
            try {
                SimpleDateFormat sdf = new SimpleDateFormat("yyyy-
MM-dd");
                sdf.setLenient(false);

                System.out.print("Enter Start Date (YYYY-MM-DD): ");
                String startDateStr = scanner.nextLine();
                System.out.print("Enter End Date (YYYY-MM-DD): ");
                String endDateStr = scanner.nextLine();

                Date startDate = sdf.parse(startDateStr);
                Date endDate = sdf.parse(endDateStr);

                java.sql.Date sqlStartDate = new
java.sql.Date(startDate.getTime());
```

```
                java.sql.Date sqlEndDate = new
java.sql.Date(endDate.getTime());


                Collection<Incident> incidents =
service.getIncidentsInDateRange(sqlStartDate, sqlEndDate);


                if (incidents.isEmpty()) {
                    System.out.println("No incidents found in the given
date range.");
                } else {
                    for (Incident inc : incidents) {
                        System.out.println("Incident ID: " +
inc.getIncidentID() +
                                ", Type: " + inc.getIncidentType() +
                                ", Date: " + inc.getIncidentDate() +
                                ", Location: (" + inc.getLatitude() + ", "
+ inc.getLongitude() + ")" +
                                ", Status: " + inc.getStatus());
                    }
                }
            } catch (ParseException e) {
                System.out.println("Invalid date format! Please enter the
date in YYYY-MM-DD format.");
            } catch (SQLException e) {
                System.out.println("Database error: " + e.getMessage());
            }
            break;
```

```
            case 4:
                try {
                    System.out.print("Enter Incident Type to Search: ");
                    String searchType = scanner.nextLine();

                    Collection<Incident> incidents =
service.searchIncidents(searchType);

                    if (incidents.isEmpty()) {
                        System.out.println("No incidents found for type: " +
searchType);
                    } else {
                        System.out.println("Incidents found:");
                        for (Incident incident : incidents) {
                            System.out.println(incident);
                        }
                    }
                } catch (SQLException e) {
                    System.out.println("Database Error: " + e.getMessage());
                }
                break;

            case 5:
                try {
                    System.out.print("Enter Report ID: ");
```

```
                int reportID = scanner.nextInt();
                scanner.nextLine();

                System.out.print("Enter Incident ID for Report: ");
                int reportIncidentID = scanner.nextInt();
                scanner.nextLine();

                System.out.print("Enter Officer ID: ");
                String reportOfficerID = scanner.nextLine();

                System.out.print("Enter Report Details: ");
                String reportDetails = scanner.nextLine();

                // Create an Incident object with necessary details
                Incident reportIncident = new Incident(reportIncidentID,
"", new Date(), 0, 0, "", "", 0, "", reportOfficerID);

                // Call the updated generateIncidentReport method
                Reports report =
service.generateIncidentReport(reportID, reportIncident, reportDetails);

                if (report != null) {
                    System.out.println("Report Generated: " + report);
                } else {
                    System.out.println("Failed to Generate Report");
                }
```

```
                } catch (SQLException e) {
                    System.out.println("Error: " + e.getMessage());
                }
                break;

            case 6:
                try {
                    System.out.print("Enter Report ID to Finalize: ");
                    int finalizeReportID = scanner.nextInt();
                    scanner.nextLine();

                    boolean finalized =
service.finalizeReport(finalizeReportID);
                    System.out.println(finalized ? "Report Finalized
Successfully!" : "Failed to Finalize Report");

                } catch (SQLException e) {
                    System.out.println("Database Error: " + e.getMessage());
                }
                break;

            case 7:
                exit = true;
                System.out.println("Exiting... Thank you!");
                break;
```

```
                default:
                    System.out.println("Invalid choice. Please try again.");
            }
        }
        scanner.close();
    }
}
```

6.Test Package:

1. CrimeAnalysisSystemTest

```
package test;
import dao.CrimeAnalysisServiceImpl;
import entity.Incident;
import org.junit.jupiter.api.*;
import
com.hexaware.myexceptions.IncidentNumberNotFoundException;
import java.sql.SQLException;
import java.util.Date;
import static org.junit.jupiter.api.Assertions.*;
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class CrimeAnalysisSystemTest {
    private CrimeAnalysisServiceImpl db;
    @BeforeAll
    void setup() {
        db = new CrimeAnalysisServiceImpl();
    }
```

```
    //Test Case 1: Incident Creation
    // Ensures an incident is created successfully with the correct
    @Test
    @DisplayName("Test: Incident is created successfully")
    void testIncidentCreatedSuccessfully() throws SQLException {
        Incident incident = new Incident(
                101, "Burglary", new Date(), 12.98, 77.59,
                "Jewelry stolen", "Open", 1, "SUS001", "OF123"
        );

        boolean result = db.createIncident(incident);
        assertTrue(result, "Incident creation should return true");

        Incident retrievedIncident = db.findIncidentById(101);
        assertNotNull(retrievedIncident, "Incident should exist in the
database");
        assertAll("Incident attributes should match",
                () -> assertEquals(101, retrievedIncident.getIncidentID(),
"Incident ID should match"),
                () -> assertEquals("Burglary",
retrievedIncident.getIncidentType(), "Incident type should match"),
                () -> assertEquals("Open", retrievedIncident.getStatus(),
"Incident status should match")
        );
    }
```

```
    // Test Case 2: Incident Status Update
    // Ensures an incident's status is updated correctly.
    @Test
    @DisplayName("Test: Incident status is updated successfully")
    void testUpdateIncidentStatusSuccessfully() throws SQLException, IncidentNumberNotFoundException {
        db.createIncident(new Incident(102, "Robbery", new Date(), 15.50, 78.40,
                "Bank heist", "Open", 2, "SUS002", "OF456"));
        boolean updateResult = db.updateIncidentStatus(102, "Closed");
        assertTrue(updateResult, "Incident status update should return true");

        Incident updatedIncident = db.findIncidentById(102);
        assertNotNull(updatedIncident, "Incident should exist in the database");
        assertEquals("Closed", updatedIncident.getStatus(), "Incident status should be updated to Closed");
    }
}
```

OUTPUTS:

```
Crime Analysis System
1. Create Incident
2. Update Incident Status
3. Get Incidents in Date Range
4. Search Incidents by Type
5. Generate Incident Report
6. Finalize Report
7. Exit
Enter your choice: 1
Enter Incident ID: 13
Enter Incident Type: Theft
Enter Latitude: 34.900702
Enter Longitude: -12.34567
Enter Description: Hand bag stolen in railway station
Enter Status: Open
Enter Victim ID: 12
Enter Suspect ID: SUS004
Enter Officer ID: OF001
Incident Created Successfully!
```

```
Crime Analysis System
1. Create Incident
2. Update Incident Status
3. Get Incidents in Date Range
4. Search Incidents by Type
5. Generate Incident Report
6. Finalize Report
7. Exit
Enter your choice: 2
Enter Incident ID to Update: 13
Enter New Status: Under Investigation
Incident Status Updated!
```

```
Crime Analysis System
1. Create Incident
2. Update Incident Status
3. Get Incidents in Date Range
4. Search Incidents by Type
5. Generate Incident Report
6. Finalize Report
7. Exit
Enter your choice: 3
Enter Start Date (YYYY-MM-DD): 2024-01-03
Enter End Date (YYYY-MM-DD): 2025-04-11
Incident ID: 12, Type: Child Abuse, Date: 2025-04-11 00:00:00.0, Location: (-33.86882, 151.20929), Status: Open
Incident ID: 13, Type: Theft, Date: 2025-04-11 00:00:00.0, Location: (34.900702, -12.34567), Status: Under Investigation
Incident ID: 21, Type: Theft, Date: 2024-02-10 14:30:00.0, Location: (13.0827, 80.2707), Status: Closed
Incident ID: 22, Type: Robbery, Date: 2024-01-20 21:15:00.0, Location: (18.5204, 73.8567), Status: Under Investigation
Incident ID: 24, Type: Burglary, Date: 2024-03-12 02:45:00.0, Location: (12.9716, 77.5946), Status: Open
Incident ID: 25, Type: Fraud, Date: 2024-01-30 10:00:00.0, Location: (22.5726, 88.3639), Status: Under Investigation
Incident ID: 26, Type: Kidnapping, Date: 2024-02-25 18:00:00.0, Location: (19.076, 72.8777), Status: Open
Incident ID: 28, Type: Cybercrime, Date: 2024-01-10 16:15:00.0, Location: (17.385, 78.4867), Status: Under Investigation
Incident ID: 29, Type: Drug Possession, Date: 2024-03-01 12:00:00.0, Location: (9.9252, 78.1198), Status: Open
Incident ID: 30, Type: Hit and Run, Date: 2024-02-17 20:45:00.0, Location: (26.8467, 80.9462), Status: Closed
```

```
Crime Analysis System
1. Create Incident
2. Update Incident Status
3. Get Incidents in Date Range
4. Search Incidents by Type
5. Generate Incident Report
6. Finalize Report
7. Exit
Enter your choice: 4
Enter Incident Type to Search: Theft
Incidents found:
Incident ID: 13, Type: Theft, Date: 2025-04-11, Location: (34.900702, -12.34567), Description: Hand bag stolen in railway station, Status: Under Investigation, Victim ID: 12, Susp
ect ID: SUS004, Officer ID: OF001
Incident ID: 21, Type: Theft, Date: 2024-02-10, Location: (13.0827, 80.2707), Description: Mobile phone stolen in a crowded bus stop., Status: Closed, Victim ID: 11, Suspect ID: S
US001, Officer ID: OF001
```

```
Crime Analysis System
1. Create Incident
2. Update Incident Status
3. Get Incidents in Date Range
4. Search Incidents by Type
5. Generate Incident Report
6. Finalize Report
7. Exit
Enter your choice: 5
Enter Report ID: 111
Enter Incident ID for Report: 12
Enter Officer ID: OF002
Enter Report Details: Allegation of mistreatment were reported regarding child abuse case on 2025-04-11.The case is currently under investigation, and futher details are awaited f
rom the assigned officer.
Report Generated: Report{reportID=111, incidentID=12, reportingOfficer='OF002', reportDate=2025-04-11T10:22:38.106866800, reportDetails='Allegation of mistreatment were reported r
egarding child abuse case on 2025-04-11.The case is currently under investigation, and futher details are awaited from the assigned officer.', status='Draft'}
```

```
Crime Analysis System
1. Create Incident
2. Update Incident Status
3. Get Incidents in Date Range
4. Search Incidents by Type
5. Generate Incident Report
6. Finalize Report
7. Exit
Enter your choice: 6
Enter Report ID to Finalize: 111
Report Finalized Successfully!
```

```
Crime Analysis System
1. Create Incident
2. Update Incident Status
3. Get Incidents in Date Range
4. Search Incidents by Type
5. Generate Incident Report
6. Finalize Report
7. Exit
Enter your choice: 7
Exiting... Thank you!
```